

CS 577 Deep Learning

Homework 1: Tools and Tensor Fluency

Assignment Specification and Grading Rubric

Fall 2026

Release:	Monday, August 17, 2026
Due:	Sunday, August 30, 2026, 11:59 p.m. Central Time
Weight:	5% of the course grade
Expected effort:	5–7 focused hours
Individual work:	This homework must be completed and submitted individually.

Learning Goals

After completing this homework, you should be able to:

1. produce a clean mathematical PDF from a LaTeX source file;
2. reason about array/tensor shapes, broadcasting, vectorization, dtype, and device;
3. translate the same computation between NumPy and PyTorch;
4. define and train a minimal `torch.nn.Module`;
5. compare an analytic gradient, a centered finite-difference approximation, and PyTorch autograd; and
6. make a small computational result reproducible.

This is an onboarding assignment, not an early workload filter. Ask questions when the software environment, notation, or submission workflow is unclear.

Files to Submit

Submit one ZIP file named `hw1_<hawk_username>.zip` containing exactly:

1. `hw1_<hawk_username>.pdf` — compiled from the provided LaTeX template;
2. `hw1_<hawk_username>.ipynb` — the completed notebook with all outputs visible; and

3. `hw1_utils.py` — your completed implementation.

The provided `test_hw1_public.py` file is a local smoke-test driver. Run it from the HW1 folder with

```
python -m unittest -v test_hw1_public.py.
```

It is not part of the three-file submission, and passing it does not guarantee full credit.

Before submitting, restart the notebook kernel and run all cells from top to bottom. The notebook must finish without an uncaught exception on the CPU path.

Collaboration, References, and Generative AI

You may discuss course concepts and general debugging strategies, but you must independently write, implement, test, and submit your work. Do not share code, completed equations, numerical answers, or solution-producing prompts.

Course materials and official NumPy, PyTorch, Python, and LaTeX documentation are allowed. Generative-AI tools may be used for conceptual explanations, syntax reminders, documentation search, or debugging suggestions. They may not produce the core derivations, implementations, experiment analysis, or written answers submitted for credit.

Every submission must include an assistance/AI disclosure. A valid disclosure names the tool or person, the purpose of the assistance, and what you verified or changed. If none was used, write: `No external assistance or generative-AI tools were used.`

Part 0: Environment and Reproducibility (10 points)

0.1 Environment Report (4 points)

In the notebook, print:

- Python, NumPy, and PyTorch versions;
- operating system;
- PyTorch device used; and
- default NumPy and PyTorch floating-point dtypes.

0.2 Seed Control (4 points)

Implement `set_all_seeds(seed)` in `hw1_utils.py`. It must set Python's `random`, NumPy, and PyTorch seeds. If CUDA is available, seed all CUDA devices as well.

Demonstrate that resetting the same seed reproduces one NumPy random vector and one PyTorch random tensor.

0.3 Reproducibility Note (2 points)

In 3–5 sentences, explain why setting a seed is useful but does not guarantee bit-for-bit reproducibility across every machine, device, library version, and parallel kernel.

Part 1: LaTeX and Mathematical Communication (15 points)

Complete `hw1_written_template.tex` and compile it to PDF.

1.1 Tensor Notation (5 points)

Typeset the following statement cleanly, including dimensions:

For a batch X with shape (B, D) , weights W with shape (D, K) , and bias b with shape $(K,)$, the affine output is

$$Y = XW + \mathbf{1}_B b^\top$$

and has shape (B, K) .

Here, $\mathbf{1}_B \in \mathbb{R}^{B \times 1}$ denotes the all-ones column vector. Therefore, $\mathbf{1}_B b^\top \in \mathbb{R}^{B \times K}$ repeats the bias vector $b^\top$ once for each of the B examples. This is the mathematical form of broadcasting b across the batch dimension in the NumPy/PyTorch expression `Y = X @ W + b`.

Use mathematical notation rather than copying the monospace statement verbatim.

1.2 Regularized Mean-Squared Error (5 points)

Typeset the objective

$$J(w) = \frac{1}{N} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

and its gradient with respect to w .

1.3 Results Table and Interpretation (5 points)

Create a LaTeX table containing:

- initial training loss;
- final training loss;
- learned slope;
- learned intercept; and
- maximum absolute difference among the analytic, finite-difference, and autograd gradients.

Add a 3–5 sentence interpretation. A table without explanatory prose does not receive full credit.

Part 2: NumPy Tensors and Vectorization (25 points)

Use the following fixed arrays:

```
X = np.array([
    [ 1.0,  2.0, -1.0],
    [ 0.0,  1.0,  3.0],
    [ 2.0, -2.0,  1.0],
    [-1.0,  0.0,  2.0],
])
```

```
W = np.array([
    [ 0.5, -1.0],
    [ 1.5,  0.0],
    [-0.5,  2.0],
])
```

```
b = np.array([0.25, -0.75])
```

2.1 Shapes and Affine Transformation (7 points)

Before running the computation, state the shapes of X , W , b , and $Y = XW + b$. Explain exactly how NumPy broadcasts b . Implement `affine_numpy(X, W, b)` without a Python loop and report Y .

2.2 Columnwise Standardization (7 points)

Implement `standardize_columns(X)` using the population standard deviation (`ddof=0`). It must return $(Z, \text{mean}, \text{std})$, where each nonconstant column of Z has mean approximately zero and standard deviation approximately one. Handle a constant column without division by zero.

2.3 Pairwise Squared Distances (8 points)

Implement `pairwise_squared_distances(A, B)` without a loop over rows. If A has shape (M, D) and B has shape (N, D) , return a matrix $D2$ with shape (M, N) where

$$D2[i, j] = \sum_k (A[i, k] - B[j, k])^2.$$

Use:

```
A = np.array([[0., 0.], [1., 0.], [0., 2.]])
B = np.array([[1., 1.], [-1., 0.]])
```

Report the resulting matrix and explain the role of the two inserted singleton dimensions.

2.4 Vectorization Timing (3 points)

Compare a clear loop implementation with the vectorized implementation on moderate random arrays. Report array sizes, the timing method, and the median of at least five runs. Do not claim a universal speedup from one machine; explain what your local result shows.

Part 3: PyTorch Tensors and a Minimal Model (25 points)

3.1 NumPy-to-PyTorch Agreement (5 points)

Convert the fixed X , W , and b arrays from Part 2 to `torch.float64` tensors and compute the affine output. Report the maximum absolute difference from the NumPy result. It should be near numerical precision.

3.2 Dtype and Device (4 points)

Show how to move the tensors to the selected device while preserving their intended dtype. In 2–4 sentences, explain why mixing devices or incompatible dtypes can cause an error or an unintended conversion.

3.3 Minimal Linear Regressor (8 points)

Implement `LinearRegressor` as a subclass of `torch.nn.Module` with one `nn.Linear(1, 1)` layer. Store the layer as `self.linear`. The forward method must accept an input tensor with shape $(B, 1)$ and return $(B, 1)$.

3.4 Deterministic Training Loop (8 points)

Use

```
x_train = torch.linspace(-1, 1, 41).reshape(-1, 1)
y_train = 2.5 * x_train - 0.4
```

Initialize the model's weight and bias to zero. Train for 100 steps using full-batch SGD with learning rate 0.1 and mean-squared error. Implement `train_tiny_regressor` and report the initial loss, final loss, learned weight, and learned bias. Plot or tabulate the loss at several iterations.

Part 4: Three Views of the Same Gradient (20 points)

Use

```
Xg = np.array([[1.0, -1.0], [0.5, 2.0], [-2.0, 1.0]])
yg = np.array([1.0, 0.0, -1.0])
w0 = np.array([0.2, -0.3])
lam = 0.1
```

and

$$J(w) = \text{mean}((X_g w - y_g)^2) + 0.5\lambda \sum_i w_i^2.$$

4.1 Analytic Gradient (7 points)

Derive the gradient in the written PDF and implement both `mse_l2_objective_numpy` and `mse_l2_grad_numpy`.

4.2 Centered Finite Differences (6 points)

A centered finite difference estimates one component of the gradient by evaluating the objective a small distance on *both sides* of the current parameter value. For component i ,

$$[\nabla J(w)]_i \approx \frac{J(w + \epsilon e_i) - J(w - \epsilon e_i)}{2\epsilon}, \quad \epsilon = 10^{-5},$$

where e_i is a vector that is zero everywhere except for a one in position i . It is called *centered* because the two evaluations are symmetric around the original w_i . This is different from perturbing every component of w at the same time.

Implement `finite_difference_gradient` as follows:

1. Create an output vector with the same shape as w .
2. For each index i , create independent copies `w_plus` and `w_minus`.
3. Change only component i : set `w_plus[i] += eps` and `w_minus[i] -= eps`.
4. Evaluate the objective at both copies and store their difference divided by `2*eps` in component i of the output.

For the supplied two-element `w0`, this requires four objective evaluations: two for each component. The original input w must be unchanged when the function returns. Do not call the analytic-gradient function or PyTorch autograd inside this finite-difference implementation.

The result is an approximation: its small discrepancy from the analytic and autograd gradients comes from finite-step truncation and floating-point rounding. With the specified `float64` computation and $\epsilon = 10^{-5}$, the maximum absolute difference from the other gradients should be below the tolerance stated at the end of this document.

4.3 Autograd Comparison (7 points)

Recreate the objective in PyTorch using `torch.float64`, call `.backward()`, and compare all three gradients. Report the vectors and the maximum absolute pairwise difference. Briefly explain why the finite-difference result is close but not necessarily identical.

Part 5: Reflection and Submission Quality (5 points)

Answer each question in 2–4 sentences:

1. Which shape or broadcasting mistake was easiest to make, and how did you detect it?
2. Which gradient validation method would you use while developing a new layer, and why?
3. What would you change in your workflow before HW2, where you will implement an MLP forward and backward pass?

The final 5 points also include readable code, meaningful variable names, visible notebook outputs, a clean PDF, and the required disclosure.

CS 577 Homework 1 Rubric

Part	Points	Full-credit evidence
0. Environment and reproducibility	10	Complete environment report; correct seed function; accurate limitation note
1. LaTeX and communication	15	Correct notation and gradient; clean PDF; well-formatted results table; interpretation
2. NumPy and vectorization	25	Correct shapes, broadcasting explanation, stable standardization, vectorized distances, honest timing
3. PyTorch and minimal model	25	NumPy agreement, correct dtype/device handling, valid <code>nn.Module</code> , deterministic training result
4. Gradient comparison	20	Correct analytic derivation and implementation; centered finite difference; autograd agreement and explanation
5. Reflection and submission quality	5	Specific reflection; readable code; visible outputs; correct filenames; complete disclosure
Total	100	

Common Deductions

- Notebook does not run from a clean restart: up to -15 , depending on severity.
- Core implementation replaced by prohibited external/AI-generated work: handled under the course academic-honesty policy.
- Missing PDF, notebook, or Python file: up to the value of the affected component.
- Hidden or missing outputs: -1 to -5 .
- Loops used where vectorization is explicitly required: up to half credit for that function.
- Correct number with no reasoning where reasoning is requested: at most half credit for that subpart.
- Missing assistance/AI disclosure: -2 and submission returned for correction if necessary.

Numerical Tolerances Used by the Public/Reference Tests

- Standard floating-point results: `atol=1e-8`, `rtol=1e-7` unless stated otherwise.
- Gradient comparison: maximum absolute error below 10^{-7} for analytic vs. autograd and below 10^{-6} for finite differences with $\epsilon = 10^{-5}$.
- Trained linear regressor after 100 specified steps: weight within 5×10^{-3} of 2.5, bias within 5×10^{-3} of -0.4 , and final loss below 5×10^{-6} .